BIG DATA · Academic year 2026-2027

Assignment 3
Thread Count, Core Sharing and Memory Bandwidth in a Shared-Memory Matrix Multiplication with OpenMP

Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria

Carlos Falcón Brito

Repository: https://github.com/Cafabri/Assigment3

Abstract

This study measures how the number of OpenMP threads changes the time of two C dense multiplication kernels reused from Assignment 2: a row-wise `ikj` loop and a cache-blocked `ikj`. Each parallel kernel runs the loops of its sequential counterpart on a contiguous range of rows per thread and is bit-identical to it. Both were timed on an Intel Core i7-13700KF (8 P-cores with two logical processors each, plus 8 E-cores) for $n = 100$ to 4000, $p = 1$ to 16 pinned threads, a scalar and an AVX2 build (median of 15 runs). With one thread per physical core the blocked kernel reached $S(8) = 7.88$–7.95 (efficiency 0.98–1.01) for $n \geq 1000$. Placing a second thread on each core made it slower ($S(16) = 5.75$–6.07), and $p = 12$ was slower still because the equal row split leaves the threads that share a core behind. The `ikj` kernel stopped at $S(8) \approx 5$ once B no longer fit in the L3 cache ($n \geq 2000$), but gained from the second logical processor ($S(16) \approx 9$). AVX2 multiplied the blocked throughput by 2.7–2.9 at every $p \leq 8$, up to 201 GFLOP/s, while its gain on `ikj` shrank as threads were added. Parallel execution helped when each core had its own arithmetic units and the work per call was large; it helped less when threads shared a core or the memory bandwidth.

1 Introduction and objective

Assignments 1 and 2 measured sequential kernels. In Assignment 2 the cache-blocked `ikj` kernel was the fastest sequential method (1.77 s at $n = 2000$ in the scalar build, 0.65 s with AVX2), and `ikj` without blocking was limited by memory reads once B left the L3 cache. A multicore processor can split the n^3 multiply-adds among cores, but the cores share the L3 cache and the memory bandwidth, the two logical processors of a core share its execution units, and the cores of this processor are not all equal. This study asks:

> *How does the number of threads affect the time, speedup, efficiency and memory of an OpenMP matrix multiplication compared with its equivalent sequential kernel, and under which conditions (matrix size, kernel, core sharing, vectorisation) does parallel execution help?*

Factors varied: thread count p, matrix size n, kernel (`ikj`, blocked), build (scalar, AVX2) and, in two extensions, the placement of the threads on the logical processors and the set of processors available (P-cores only, or P- and E-cores). **Outcomes:** kernel time, speedup, efficiency and the peak memory of each process. **Kept fixed:** language (C), compiler, algorithm and storage layout of each kernel, float64 values, input matrices, machine.

Predictions. Table 1 lists the predictions. They were committed to the repository (`report/hypotheses.md`) while the measurement run was in progress and before its re-

sults were analysed. H5 was formulated after a short smoke test of the runner ($n \leq 250$) showed a drop at $p = 12$, so it is informed by a pilot, not blind.

Id	Prediction (reason)
H1	Blocked, $n \geq 1000$: $E(p) \geq 0.8$ up to $p = 8$ and $S(16)/S(8) \leq 1.2$ (limited by arithmetic; the two logical processors of a core share its floating-point units)
H2	`ikj`, $n \geq 2000$: $S(8)$ clearly below 8 and below that of blocked, and $S(16) > S(8)$ (B no longer fits in the L3 and is streamed from memory for every row of C; a second thread per core can hide part of the memory latency)
H3	$n = 100$: $S(p) < 8$ for every p (fork/join and imbalance of 100 rows are not negligible against 0.1–0.3 ms of work)
H4	$T(\text{OpenMP}, p = 1)/T_{seq}$ within 2% of 1 for $n \geq 250$ (same loops; only the region entry and the row range are added)
H5	(pilot-informed) $E(12) < E(8)$ for both kernels (with 12 threads four cores run two threads and four run one; equal row ranges make the threads sharing a core finish last)
H6	AVX2/scalar time ratio of blocked about constant in p (2–3×); that of `ikj` decreasing with p at $n \geq 2000$ (SIMD does not increase memory bandwidth)
H7	Peak working set $\approx 24n^2$ bytes plus a constant, growing by less than 1 MiB per thread (the threads share A, B and C)
H8	All 24 logical processors, threads placed by Windows: $S(24) \leq 1.1\,S(16)$, and $S(32)$, $S(48) < S(24)$ for blocked (slower E-cores with an equal split; time-slicing)

Table 1: Predictions, evaluated in Section 5.1 from `results/report_numbers.txt`.

2 Background

Shared-memory parallelism and OpenMP. OpenMP [1] expresses parallelism with compiler directives: a `parallel` region is executed by a team of threads (fork) that wait for each other at its end (join). The GNU implementation, libgomp [2], keeps its threads alive between regions, so after the first region a fork/join costs microseconds, not the creation of threads. The multiplication $C = AB$ parallelises without synchronisation when each thread computes different rows of C: A and B are only read, and no element of C is written by two threads.

Speedup models. With T_{seq} the time of the sequential program and T_p that of the parallel one with p threads, the speedup is $S(p) = T_{seq}/T_p$ and the efficiency $E(p) = S(p)/p$. If a fraction f of the work is serial, Amdahl's law bounds $S(p) \leq 1/(f + (1 - f)/p)$ [3]. The Karp–Flatt metric $e = (1/S - 1/p)/(1 - 1/p)$ is the value of f that would explain an observed speedup; if e grows with p, the loss comes from overheads that grow with p (imbalance, contention), not from a fixed serial part [4].

Resources shared by threads. A kernel is *compute-bound* when the arithmetic units limit it and *memory-bound* when the bandwidth or latency of memory does; the roofline model places a kernel by its operations per byte moved [5]. The blocked kernel reuses $b \times b$ tiles from the cache [6] and is compute-bound; `ikj` reads all of B ($8n^2$ bytes) for every row of C, which is cheap while B fits in the cache and limited by memory when it does not. Adding cores adds arithmetic units and private caches (L1, L2) but not memory bandwidth or L3 capacity. *Simultaneous multithreading* (SMT, Hyper-Threading) runs two threads on one core: they share its execution units and L1/L2 caches, which helps when one thread stalls on memory and can hurt when both need the same units or overflow the shared cache [7, 8].

Hybrid processor and vectorisation. The i7-13700KF combines 8 P-cores (with SMT) and 8 slower E-cores (without SMT) whose placement Windows decides with hints from the hardware [9]; a thread can be restricted to a logical processor with `SetThreadAffinityMask` [10].

Vectorisation executes one instruction on 4 doubles (AVX2); gcc vectorises loops whose iterations are independent, here the innermost j loop of both kernels [11].

Rounding. Floating-point addition is not associative [12]. All kernels add the n products of each $C[i][j]$ in increasing k, so splitting rows among threads does not change any sum and the results are bit-identical.

3 Methodology

3.1 Implementations and reused material

The sequential baselines are the kernels of Assignment 2 (`https://github.com/Cafabri/Assigment2`, commit `c40444e`, `c/matmul.c`), copied unchanged (`c/matmul.c`, checked with `diff`): `ikj` (j innermost, unit-stride rows of B and C) and **blocked** (`ikj` on $b \times b$ tiles, tile loops ii, kk, jj). The block sizes are those chosen in Assignment 2 for each build ($b = 32$ scalar, $b = 256$ AVX2) and were not tuned again. Two kernels were kept on purpose: one compute-bound and one that becomes memory-bound, so that the study can show when threads help. The SplitMix64 inputs, the timer and the batching rule are also those of Assignment 2.

The parallel versions (`c/parallel.c`) open one `omp parallel` region with exactly p threads (`num_threads`; `omp_set_dynamic(0)`). Thread t computes the rows $[i_0, i_1)$ of a contiguous split in which the range sizes differ by at most one row, and runs on them exactly the loops of its sequential kernel (for blocked, the row tiles start at i_0, so a range can end with a shorter tile). The work is divided statically and equally, and there is no lock, atomic or reduction. All code is my own; no numerical library is called by any kernel.

3.2 Input data

A and B are dense $n \times n$ float64 matrices from the counter-based SplitMix64 generator of Assignment 1 (seed 42 for A, 43 for B, values uniform in $[0, 10)$, position $p = i \cdot n + j$), row-major. They are the matrices of Assignments 1 and 2, so the checksums can be compared across assignments. All values are non-negative, so no cancellation occurs.

3.3 Environment

CPU: Intel Core i7-13700KF; P-cores 0–7 with logical processors $\{0, 1\}$, $\{2, 3\}$, …, $\{14, 15\}$ (read with `GetLogicalProcessorInformationEx`), E-cores on logical processors 16–23; 48 KB L1D and 2 MB L2 per P-core, 30 MB shared L3; 31.8 GiB RAM; Windows 11 Pro, power plan *Balanced*, frequency not fixed. **Compiler:** gcc 16.1.0 (MSYS2/MinGW64) with its libgomp; build `scalar`: `-O2 -ffp-contract=off -fopenmp` (the flags of Assignment 2 plus OpenMP); build `avx2`: adds `-fvect-cost-model=dynamic -mavx2`. **Evidence of vectorisation:** the gcc report (`-fopt-info-vec`, `results/vectorisation_report.txt`) lists no vectorised kernel loop in the scalar build and, in the AVX2 build, the innermost j loop of both kernels, sequential and parallel (`matmul.c` lines 24 and 43, `parallel.c` lines 55 and 73), with 32-byte vectors behind a run-time overlap check. **Analysis:** CPython 3.11.9, NumPy 2.2 (reference results), pandas 2.2, matplotlib 3.10.

Thread control. The runner restricts its process, and so every benchmark process, to the 16 logical processors of the P-cores. Inside the program, OpenMP thread t pins itself with `SetThreadAffinityMask` to entry t of a placement list: **spread** $= (0, 2, 4, \ldots, 14, 1, 3, \ldots, 15)$, one thread per physical core before any core gets a second one, used in the main experiment; **compact** $= (0, 1, 2, \ldots, 15)$, two threads per core, used only in an extension. The sequential kernel runs on the main thread, pinned like thread 0 to logical processor 0. Before timing, a parallel region checks that the team has exactly p threads; each thread also records the processor it runs on (`results/placement.csv`). No numerical library is linked, so no library creates threads of its own; `OMP_PROC_BIND`, `OMP_PLACES` and `OMP_WAIT_POLICY` are unset (libgomp defaults) and

`OMP_DYNAMIC=false`. Oversubscription happens only in the hybrid extension ($p = 32, 48 > 24$ logical processors).

3.4 Experiments and measurement protocol

Main experiment (`threads`). Both builds, both kernels, the sequential kernel and its OpenMP version with $p \in \{1, 2, 4, 6, 8, 12, 16\}$, spread placement, $n \in \{100, 250, 500, 1000, 2000, 4000\}$: 192 configurations. The size sequence was fixed before running; it ends at $n = 4000$ ($24n^2 = 384$ MB) so that the sequential `ikj` (28 s per run) can be repeated 15 times. **Placement extension:** compact placement, $p \in \{2, 4, 8, 16\}$, scalar build, $n \in \{1000, 4000\}$. **Hybrid extension:** the process may use all 24 logical processors and the threads are not pinned (Windows places them), $p \in \{1, 8, 16, 24, 32, 48\}$ and the sequential kernels, scalar build, $n \in \{1000, 4000\}$. This changes two factors at once (available processors and pinning), so it is compared with the main experiment only qualitatively.

Timer. `QueryPerformanceCounter` (monotonic) around the kernel call only. A and B are generated and C allocated and zeroed outside it. For a parallel kernel the timed call includes entering and leaving the parallel region (fork/join) and the computation of the row range, but not the creation of the threads, which happens in the untimed check region. Every unbatched run writes the checksum of C, so the computation cannot be removed.

Repetitions. Each benchmark process makes 2 warm-up runs and 5 measured runs; each configuration runs in 3 independent processes, interleaved (all configurations once, then the second and third process of each), so that slow drifts affect all configurations alike: 15 measured runs per configuration. A first call shorter than 10 ms is a separate check call and the runs are timed in batches of at least 10 ms, reporting the time per call (only at $n \leq 500$; up to 512 calls per batch at $n = 100$). Every raw time is kept (`results/raw_measurements.csv`).

Budget. 15 minutes per run (otherwise the process is killed: TIMEOUT) and 16 GiB of matrix storage per process (checked before allocating), as in Assignments 1 and 2.

Metrics. T_{seq} and T_p are the *medians* of the 15 measured runs of the sequential kernel and of its OpenMP version (same build, n and experiment); $S(p) = T_{seq}/T_p$ and $E(p) = S(p)/p$. The 95% interval of $S(p)$ is a percentile bootstrap [14]: the runs of both configurations are resampled independently (2000 resamples, fixed seed) and the ratio of medians recomputed. Variability is the IQR of the 15 runs. Throughput is $2n^3/T$ (GFLOP/s).

Memory. The runner reads the *peak working set* and the *peak private bytes* of each benchmark process with `GetProcessMemoryInfo` after it exits; the *storage* of the matrices is computed, $24n^2$ bytes. These are different quantities and are reported separately.

3.5 Correctness

(1) At start-up every program checks identity, zero and 2×2 cases and a 37×37 generated input with $b = 5$ (partial tiles, row ranges not multiple of b): both sequential kernels and both parallel kernels with $p \in \{1, 2, 3, 4, 7, 8, 16, 24, 40\}$ (more threads than rows) must equal `ijk` bit for bit. (2) For $n \in \{1, 7, 100, 333, 500\}$, every kernel, build and $p \in \{1, 2, 3, 8, 16, 24\}$ writes its full result, which is compared, on inputs regenerated independently in NumPy, with an exact NumPy emulation that adds the outer products $A[:, k]\, B[k, :]$ in increasing k (must be bit-identical) and with `numpy.matmul` (relative tolerance nu, with $u = 2^{-53}$, from the bound $|\hat{C} - C| \leq \gamma_n |A||B|$ and $|A||B| = AB$ for non-negative inputs [13]). (3) The checksum of every unbatched run must take a single value per n across methods, builds, thread counts and placements, and equal the checksum of Assignment 2 for the same n.

4 Results

4.1 Correctness, completeness and variability

All 708 benchmark processes ended OK (no timeout, memory limit or error; 157 min), so the largest tested size, $n = 4000$, is the largest attempted, within the budget for every configuration. All 140 element-wise checks were bit-identical to the exact reference; the largest difference from `numpy.matmul` was 3.2×10^{-15} (tolerance 5.6×10^{-14} at $n = 500$). For each n a single checksum was observed over 32–54 configurations, equal to that of Assignment 2 at $n = 100$, 1000, 2000 and 4000. Every pinned thread ran on its requested processor.

Table 2 summarises the IQR. For $n \geq 1000$ it is at most 3% of the median in 82% of the configurations; at $n = 4000$ its median is 0.45%. The short calls at $n = 100$ (30–250 μs) vary most (up to 29%). The sequential `ikj` at $n = 1000$ differed by 8% between processes (medians 0.243–0.262 s), the largest process spread among the baselines.

n	Runs	Calls per batch	Median IQR (% of median)	Largest IQR (%), configuration
100	15–15	64–512	4.47	29.13 (`blocked_omp`, avx2, $p = 8$)
250	15–15	4–64	2.14	16.72 (`blocked_omp`, scalar, $p = 12$)
500	15–15	1–16	2.84	5.11 (`ikj_omp`, scalar, $p = 8$)
1000	15–15	1–1	1.62	12.41 (`ikj_omp`, avx2, $p = 16$)
2000	15–15	1–1	1.02	7.93 (`ikj_omp`, scalar, $p = 16$)
4000	15–15	1–1	0.45	4.96 (`ikj_omp`, avx2, $p = 12$)

Table 2: Variability of the 15 measured runs per configuration, main experiment.

4.2 Runtime, speedup and efficiency (scalar build)

Table 3 gives the medians and Figure 1 the speedup and efficiency. The sequential times are those of Assignment 2 within a few percent (blocked 1.80 s at $n = 2000$ against 1.77 s there).

		OpenMP, p threads (spread placement)						
n	Sequential	1	2	4	6	8	12	16
ikj								
100	252 μs	254 μs	137 μs	77.1 μs	59 μs	50 μs	60.3 μs	53.4 μs
250	3.59 ms	3.58 ms	1.8 ms	931 μs	638 μs	489 μs	576 μs	452 μs
500	29.7 ms	29.6 ms	14.8 ms	7.5 ms	5 ms	3.88 ms	4.43 ms	3.38 ms
1000	248 ms	245 ms	118 ms	59.7 ms	40 ms	29.8 ms	34.5 ms	25.9 ms
2000	2.92 s	2.91 s	1.58 s	897 ms	660 ms	575 ms	348 ms	329 ms
4000	28.1 s	28.1 s	15.2 s	8.33 s	6.07 s	5.75 s	3.59 s	3.14 s
blocked								
100	226 μs	230 μs	127 μs	70.5 μs	55.2 μs	48.2 μs	69 μs	60.5 μs
250	3.47 ms	3.46 ms	1.74 ms	919 μs	618 μs	476 μs	781 μs	606 μs
500	27.9 ms	27.7 ms	14.1 ms	7.15 ms	4.81 ms	3.63 ms	5.97 ms	4.91 ms
1000	224 ms	223 ms	112 ms	56.7 ms	37.8 ms	28.4 ms	48 ms	38 ms
2000	1.8 s	1.79 s	898 ms	456 ms	303 ms	227 ms	386 ms	296 ms
4000	14.3 s	14.2 s	7.13 s	3.62 s	2.39 s	1.8 s	3.28 s	2.49 s

Table 3: Median time per multiplication, scalar build (15 runs; IQR in `results/summary.csv`).

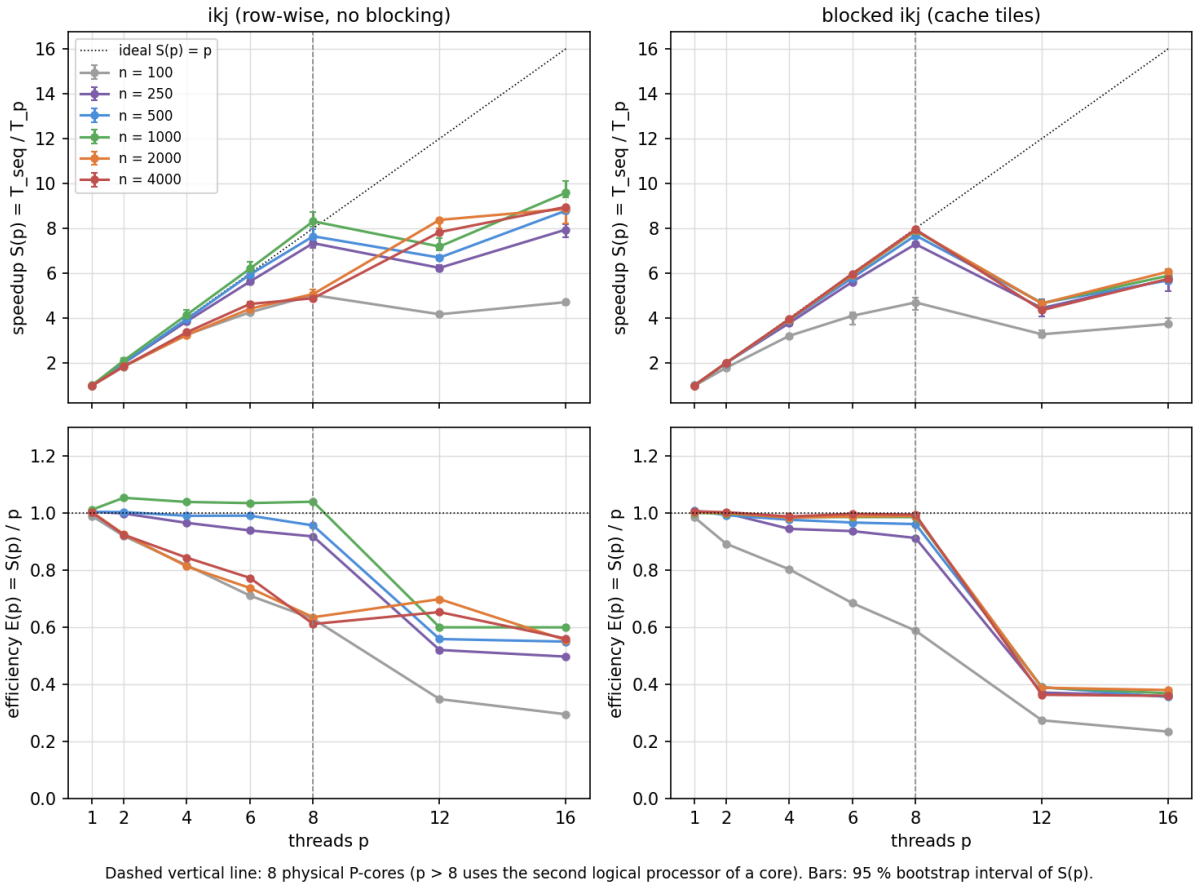


Figure 1: Speedup (top) and efficiency (bottom) against threads, scalar build, spread placement. Up to $p = 8$ every thread has its own physical core; beyond, cores get a second thread. Bars: 95% bootstrap interval.

Blocked. For $n \geq 500$ the efficiency stays at 0.96–1.01 up to $p = 8$: $S(8) = 7.88$, 7.91 and 7.95 at $n = 1000$, 2000 and 4000 (70.5–71.2 GFLOP/s against 8.9 sequential). Beyond $p = 8$ the speedup *drops*: $S(12) = 4.35$–4.66 and $S(16) = 5.75$–6.07, so 16 threads are 23–28% slower than 8. **`ikj`.** At $n \leq 1000$ it scales like blocked ($S(8) = 8.31$ at $n = 1000$). At $n = 2000$ and 4000 the efficiency falls steadily from $p = 2$ and $S(8) = 5.08$ and 4.89; here the second logical processor helps, $S(16) = 8.87$ and 8.96, and even $S(12) > S(8)$. **Small sizes.** At $n = 100$ the speedup peaks at $p = 8$ with 5.03 (`ikj`) and 4.70 (blocked), 50 μs per call.

One thread. The OpenMP kernel with $p = 1$ took 0.989–1.015 times the sequential time for every n and kernel in the scalar build (Figure 5 in the appendix), i.e. the parallel region adds no measurable cost at $n \geq 250$ and about 1.5% at $n = 100$ (blocked, 226 μs).

Superlinear points. Only scalar `ikj` at $n = 1000$ ($p = 2, 4, 8$; $S(8) = 8.31$, interval 8.08–8.74), AVX2 `ikj` at $n = 1000$, $p = 2$ (2.03) and the AVX2 blocked kernel for $p \leq 8$ (Section 4.4) have a bootstrap interval entirely above p; they are discussed in Section 5.4.

4.3 Core sharing: placement extension

Figure 2 compares spread and compact placement. With two threads on *one* core (compact, $p = 2$), blocked ran at 0.83 times the speed of one thread alone at both sizes, while `ikj` ran 1.20 ($n = 1000$) and 1.42 ($n = 4000$) times faster. Compact $p = 8$ (4 cores) gave blocked $S = 2.9$–3.1 against 7.9 with spread (8 cores). With all 16 logical processors both placements use the same processors and agree within 6%. For `ikj` at $n = 4000$ compact $p = 8$ was faster than spread $p = 8$ (5.41 against 4.89).

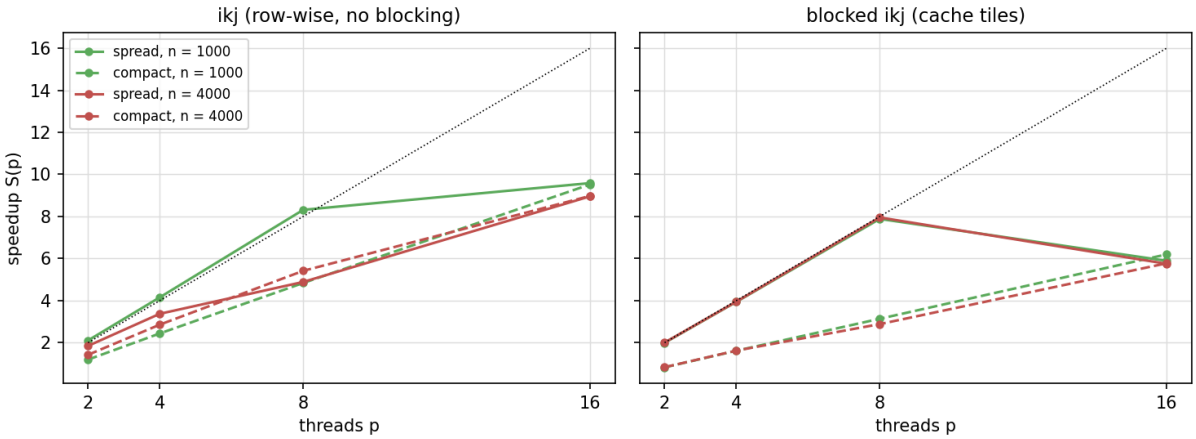


Figure 2: Spread (one thread per physical core first) against compact (two threads per core) placement, scalar build; dotted: $S = p$.

4.4 Vectorisation extension

Figure 3 shows the throughput of both builds and Table 4 the ratio of scalar to AVX2 time at equal kernel, n and p, which isolates the SIMD effect from the thread effect. For blocked the ratio is 2.73–2.86 at every $p \leq 8$, so AVX2 and threads multiply: 189.8–200.8 GFLOP/s at $p = 8$, 21× the sequential scalar blocked kernel at $n = 4000$ (14.3 s to 0.674 s). At $p = 12$ and 16 the ratio rises to 3.3–3.5, because the AVX2 blocked kernel loses less from core sharing ($S(16)/S(8) = 0.92$, against 0.72–0.77 scalar). For `ikj` at $n = 4000$ the ratio falls from 1.44 (sequential) to 1.15–1.37 for $p \geq 6$, and at $n = 2000$ from 1.68 to 1.40 at $p = 16$.

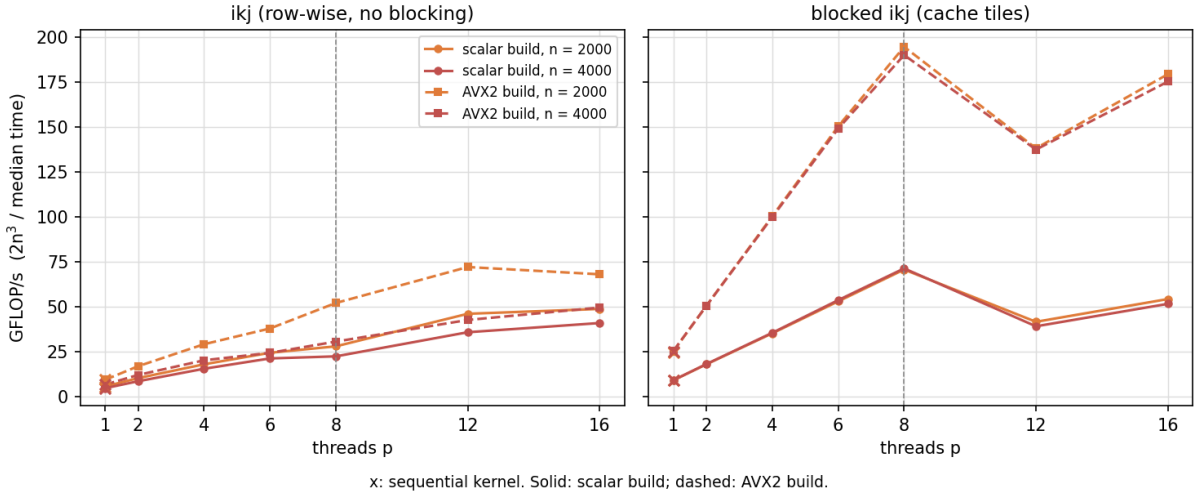


Figure 3: Throughput ($2n^3$ / median time) of the scalar and AVX2 builds at $n = 2000$ and 4000, spread placement.

			OpenMP, p threads						
Kernel	n	Sequential	1	2	4	6	8	12	16
ikj	1000	1.91	1.90	1.85	1.85	1.85	1.84	2.22	2.04
ikj	2000	1.68	1.67	1.65	1.63	1.56	1.87	1.57	1.40
ikj	4000	1.44	1.44	1.42	1.31	1.15	1.37	1.19	1.21
blocked	1000	2.73	2.80	2.80	2.82	2.78	2.85	3.34	3.51
blocked	2000	2.76	2.81	2.82	2.86	2.84	2.76	3.33	3.31
blocked	4000	2.77	2.81	2.80	2.82	2.78	2.67	3.52	3.40

Table 4: $T_{\text{scalar}}/T_{\text{AVX2}}$ (ratio of medians) at the same kernel, n and p.

In the AVX2 build the OpenMP blocked kernel with one thread was 1.9–2.5% *faster* than the sequential one at $n \geq 1000$ ($T_1/T_{seq} = 0.975$–0.981, intervals below 1). Relative to this one-thread time the speedup is never above p (at most 2.007 at $p = 2$), so the apparent superlinear speedups of this kernel come from the faster single-thread code of the parallel version, not from extra parallel gain. At $n = 100$ the AVX2 kernels gain little from threads ($S \leq 2.91$; 87–91 μs sequential). The AVX2 speedup and runtime tables are in the appendix.

4.5 Hybrid extension: E-cores and oversubscription

With all 24 logical processors available and the placement left to Windows (Figure 4), blocked reached $S(16) = 8.18$–8.52 and $S(24) = 8.83$–9.27 ($S(24)/S(16) = 1.08$–1.09) and kept growing slightly when oversubscribed: $S(48) = 9.04$–9.41. `ikj` reached $S(24) = 12.52$ at $n = 1000$ (1.29 times $S(16)$) but not at $n = 4000$ (9.46 against 9.91 at $p = 16$). With 8 threads the unpinned runs were slower than the pinned spread ones (blocked 5.99–6.78 against 7.88–7.95). A snapshot of the placement found the 8 threads on 7 distinct P-cores and, at $p = 16$, 14 threads on 7 P-cores and 2 on E-cores; at $p = 48$ all 24 logical processors were in use. These are single observations of a placement that the scheduler changes during a run.

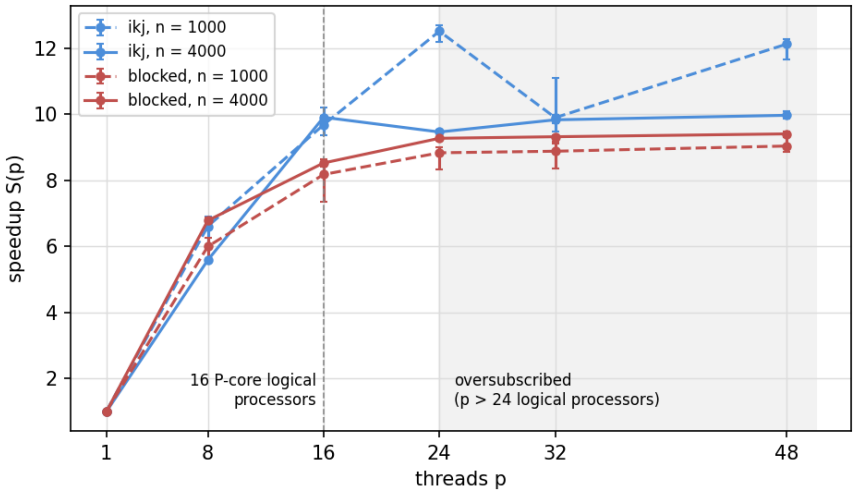


Figure 4: Hybrid extension: all 24 logical processors, threads placed by Windows, scalar build; grey: more threads than logical processors.

4.6 Memory

The peak working set exceeded the $24n^2$ bytes of A, B and C by 4.75–5.44 MiB in every configuration, sequential or parallel, at every n (Table 5): the threads share the three matrices and the constant part is the program, the C runtime and libgomp. The peak private bytes grew from $p = 2$ to $p = 16$ by 416–484 KiB at $n = 4000$, 30–35 KiB per extra thread (stacks and runtime data, mostly reserved rather than resident). The resolution of these OS counters (pages, and a sampled peak) limits differences below about 1 MiB.

			OpenMP, p threads						
Kernel	$24n^2$	Sequential	1	2	4	6	8	12	16
ikj	366.2	371.6	371.6	371.0	371.1	371.1	371.1	371.1	371.2
blocked	366.2	371.6	371.6	371.0	371.0	371.0	371.1	371.2	371.2

Table 5: Peak working set (MiB, median of 3 processes) at $n = 4000$, scalar build, and the computed storage of A, B and C.

5 Discussion

5.1 Evaluation of the predictions

Id	Outcome	Evidence
H1	Supported, stronger than predicted	$E(p \le 8) = 0.98$–1.01 (scalar), 0.96–1.03 (AVX2); $S(16)/S(8) = 0.72$–0.77 (scalar) and 0.92 (AVX2): no gain, a loss
H2	Supported	$n \ge 2000$: `ikj` $S(8) = 4.66$–5.65 against 7.66–7.95 for blocked; $S(16) = 7.39$–$8.96 > S(8)$
H3	Supported	$n = 100$: largest S 5.03 (scalar) and 2.91 (AVX2)
H4	Mostly supported	Scalar: 0.989–1.015 at every n; AVX2 `ikj`: within 1% for $n \ge 250$; AVX2 blocked: 0.975 at $n = 1000$ (the parallel code is faster), 1.16 at $n = 100$
H5	Supported for blocked, not for `ikj` at $n \ge 2000$ (scalar)	Blocked: $E(12) = 0.36$–0.48 against $E(8) = 0.96$–1.03. Scalar `ikj` $n \ge 2000$: $E(12) = 0.65$–$0.70 > E(8) = 0.61$–0.63
H6	Supported up to $p = 8$	Blocked 2.73–2.86 for $p \le 8$ (3.3–3.5 at $p \ge 12$); `ikj` $n = 4000$: $1.44 \to 1.15$–1.37
H7	Supported	Peak $- 24n^2$: 4.75–5.44 MiB for every p; +30–35 KiB per thread
H8	Partly	Blocked $S(24)/S(16) = 1.08$–1.09, but $S(48) \ge S(24)$; `ikj` $n = 1000$: $S(24)/S(16) = 1.29$

Table 6: Predictions against the measurements (`results/report_numbers.txt`).

5.2 Why the blocked kernel scales on cores but not on logical processors

Up to $p = 8$ each thread has its own core, L1 and L2, the tiles of B are reused from the private caches, and the only shared resource that grows in use (L3 and memory traffic) stays small for a tiled kernel. The efficiency close to 1 and a Karp–Flatt fraction $e \le 0.006$ for $p \le 8$ at $n \ge 1000$ show that fork/join and the serial parts are negligible there. The placement experiment isolates the second logical processor: two threads on one core gave 0.83 times the throughput of one thread. The core's floating-point units are already busy with one thread, so a second thread cannot add arithmetic; it only shares the L1 data cache (48 KB) and the L2. The three 32×32 tiles of the scalar build occupy 24 KB, so two threads need about all of the L1; the AVX2 build ($b = 256$, tiles in the L2) lost less ($S(16)/S(8) = 0.92$). This cache explanation is consistent with the two block sizes but was not measured with hardware counters.

The drop at $p = 12$ follows from the static equal split. Four cores run two threads, four run one, and every thread gets $n/12$ rows, so the run ends when the slowest threads finish. If a thread on a shared core progresses at rate r_2 (relative to a thread alone) and all threads at $p = 16$ share cores, then $S(16) \approx 16r_2$ and $S(12) \approx 12r_2 = 12\,S(16)/16$. This model, using only the measured $S(16)$, predicts $S(12) = 4.31$–4.55 (scalar) and 5.29–5.67 (AVX2) against measured 4.35–4.66 and 5.53–5.70. A dynamic schedule or an uneven split would let faster threads take more rows; this was not tested.

5.3 Why `ikj` saturates, and when SMT helps

`ikj` reads all of B for each row of C. At $n = 1000$, B (8 MB) stays in the 30 MB L3 and each core streams it at its own rate, so `ikj` scales like blocked. At $n \geq 2000$, B (32–128 MB) no longer fits and is read from memory for every row, and eight threads reach only 22.3 GFLOP/s at $n = 4000$ (against 71.2 for blocked). The Karp–Flatt fraction is already 0.06–0.09 from $p = 2$ to 8 (against $e < 0$ at $n = 1000$). The metric alone cannot separate a serial part from contention, but the parallel region of this kernel contains only the row loops, and the loss appears exactly when B leaves the L3, so the likely cause is that the threads compete for the shared memory bandwidth. Here the second logical processor helps (1.42 times one thread on one core at $n = 4000$): while one thread waits for memory, the other computes. This is also why compact $p = 8$ beat spread $p = 8$ for `ikj` at $n = 4000$: two threads on a core share its L2 and read the same rows of B at about the same time. The same limit explains H6: AVX2 shortens the arithmetic but not the wait for memory, so its gain shrinks as threads saturate the bandwidth. The memory bandwidth itself was not measured.

5.4 Small sizes, one thread and superlinear points

At $n = 100$ a call lasts 30–250 μs. Each of up to 16 threads gets 6–7 rows, and a fork/join of the pool plus the uneven end of the threads becomes a visible fraction: $S \leq 5.03$ (scalar) and ≤ 2.91 (AVX2, whose sequential call is already 3× shorter). With one thread the parallel kernel costs the same as the sequential one in the scalar build. In the AVX2 build the blocked parallel kernel is about 2% faster; its code differs only in how the row bounds are passed (`blocked_rows` receives $[i_0, i_1)$), so the compiler may lay out or align the loops differently. This was not analysed at the instruction level. The superlinear points of scalar `ikj` at $n = 1000$ are 1–4% above p, the lower bound of the interval only 1% above at $p = 8$, and coincide with the largest process spread of a baseline (8%). A plausible cause, shared reads of B in the L3 by threads that progress together, was not measured, so these points are reported as small and not explained.

5.5 E-cores and oversubscription

The hybrid runs had 8 more cores available, so their higher $S(16)$ for blocked (8.2–8.5 against 5.8–6.1 pinned) does not mean that the Windows placement is better; it means that 16 threads spread over P- and E-cores did more than 16 threads on 8 cores, each of which delivers only 0.83 of its single-thread throughput when shared. With 8 threads the unpinned runs were slower than pinned ones: the scheduler did not always use 8 distinct cores. Oversubscription did not reduce the blocked speedup, contrary to H8: with 48 threads each gets fewer rows, and threads that finish early free a processor for waiting ones, which acts like a dynamic schedule over fast and slow cores. This interpretation follows from the static row split and was not tested directly. The `ikj` results at $n = 1000$ vary strongly with p (9.90 at $p = 32$, 12.52 at $p = 24$) and have wide intervals; they are reported, not interpreted.

5.6 Answer to the research question

For these kernels and this machine, parallel execution helped most when (i) each thread had its own physical core, (ii) the kernel was limited by arithmetic (blocked, efficiency 0.96–1.03 at $p \leq 8$, $n \geq 500$, both builds) and (iii) a call took at least about a millisecond. It helped less when the threads shared the memory bandwidth (`ikj`, $n \geq 2000$: $S(8) \approx 5$), and it hurt when two compute-bound threads shared a core, alone (0.83) or mixed with unshared cores under an equal split ($p = 12$). The second logical processor of a core paid off only for the memory-bound kernel. Vectorisation was complementary for the compute-bound kernel (2.7–2.9× at every $p \leq 8$), and its gain fell as the memory-bound kernel approached the bandwidth limit. The extra memory of the threads was negligible (about 30 KiB each).

5.7 Limitations

One machine, compiler and OS; frequency not fixed (Balanced plan), so turbo frequency may fall as more cores are active, which lowers $S(p)$ and is included in the measured speedups. No hardware counters: the cache, bandwidth and SMT explanations are inferred from timings, block sizes and placement experiments. Block sizes were tuned in Assignment 2 for one thread, not for each p. Only a static equal row split was tested. The hybrid extension changes the available processors and the pinning at once, and its placement observations are snapshots. The peak-memory counters cannot resolve per-thread costs below about 1 MiB. Different machines, compilers or schedules can change all of these numbers.

6 Conclusions

Within the tested kernels, sizes ($n \leq 4000$), builds and the i7-13700KF:

- An OpenMP row split of the blocked kernel is bit-identical to the sequential kernel and scales almost ideally on physical cores: $S(8) = 7.9$ for $n \geq 1000$, with no measurable cost at $p = 1$.
- Putting a second compute-bound thread on a core lowered the throughput of the core (0.83), and mixing shared and unshared cores under an equal split made $p = 12$ the worst configuration above $p = 8$; a simple imbalance model reproduces it.
- The row-wise `ikj` kernel is limited by memory once B leaves the L3: $S(8) \approx 5$, and the second logical processor recovers part of the loss ($S(16) \approx 9$).
- AVX2 and threads combine for the compute-bound kernel (up to 201 GFLOP/s, 21× the scalar sequential kernel), but AVX2 gains less as a memory-bound kernel uses more threads.
- Threads add negligible memory: the three matrices are shared.

Open questions. Hardware-counter evidence for the SMT and bandwidth explanations; dynamic or uneven scheduling on shared and hybrid cores; block sizes tuned per thread count; the cause of the small superlinear points of `ikj` at $n = 1000$; and the distributed version of Assignment 4, which will reuse these kernels.

Reproducibility

The repository holds the code, the exact commands (`README.md`), the raw data and the analysis scripts. `python run_experiment.py` builds both binaries and runs all three experiments (157 min here); `analysis/check_correctness.py`, `plot_results.py` and `report_numbers.py` regenerate the validation, the figures, the tables and every number in this report.

Use of generative AI

Claude Opus 5.5 (Anthropic), used through Claude Code, served as a support tool during this assignment: to help plan the study, write and debug the parallel kernels, the benchmark program, the experiment runner and the analysis scripts, and to draft and revise the wording of the report. All the measurements come from running the code on the machine described in Section 3.3, and every number in the text is produced by `analysis/report_numbers.py` from the raw data. The author reviewed the code and the AI-generated text, checked the numbers against the raw data and the references, and is responsible for the conclusions.

References

[1] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, Nov. 2021. [Online]. Available: https://www.openmp.org/specifications/

[2] Free Software Foundation, *GNU Offloading and Multi Processing Runtime Library: The GNU OpenMP and OpenACC Implementation*. [Online]. Available: https://gcc.gnu.org/onlinedocs/libgomp/. Accessed: Nov. 12, 2026.

[3] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485, doi: 10.1145/1465482.1465560.

[4] A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," *Commun. ACM*, vol. 33, no. 5, pp. 539–543, May 1990, doi: 10.1145/78607.78614.

[5] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, doi: 10.1145/1498765.1498785.

[6] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, 1991, pp. 63–74, doi: 10.1145/106972.106981.

[7] D. M. Tullsen, S. J. Eggers, and H. M. Levy, "Simultaneous multithreading: Maximizing on-chip parallelism," in *Proc. 22nd Annu. Int. Symp. Computer Architecture (ISCA)*, 1995, pp. 392–403, doi: 10.1145/223982.224449.

[8] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.

[9] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*. [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html. Accessed: Nov. 12, 2026.

[10] Microsoft, "SetThreadAffinityMask function (winbase.h)," Win32 API documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-setthreadaffinitymask. Accessed: Nov. 12, 2026.

[11] Free Software Foundation, "Options that control optimization," in *Using the GNU Compiler Collection (GCC)*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html. Accessed: Nov. 12, 2026.

[12] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Comput. Surv.*, vol. 23, no. 1, pp. 5–48, Mar. 1991, doi: 10.1145/103162.103163.

[13] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[14] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

Appendix: AVX2 build and one-thread overhead

		OpenMP, p threads (spread placement)						
n	Sequential	1	2	4	6	8	12	16
ikj								
100	87.2 μs	94.2 μs	57 μs	35.1 μs	31.7 μs	30.7 μs	36.4 μs	35.6 μs
250	1.44 ms	1.45 ms	741 μs	394 μs	270 μs	211 μs	255 μs	207 μs
500	10.1 ms	10.1 ms	5.14 ms	2.58 ms	1.72 ms	1.34 ms	1.69 ms	1.3 ms
1000	130 ms	129 ms	63.8 ms	32.3 ms	21.7 ms	16.2 ms	15.5 ms	12.7 ms
2000	1.74 s	1.74 s	954 ms	552 ms	424 ms	308 ms	222 ms	236 ms
4000	19.6 s	19.6 s	10.8 s	6.38 s	5.29 s	4.2 s	3.01 s	2.59 s
blocked								
100	91 μs	106 μs	62.3 μs	39.6 μs	32.4 μs	31.2 μs	35 μs	34.5 μs
250	1.46 ms	1.47 ms	756 μs	393 μs	275 μs	217 μs	264 μs	209 μs
500	9.87 ms	9.81 ms	4.86 ms	2.53 ms	1.7 ms	1.29 ms	1.81 ms	1.4 ms
1000	81.9 ms	79.9 ms	40.1 ms	20.1 ms	13.6 ms	9.96 ms	14.4 ms	10.8 ms
2000	651 ms	638 ms	318 ms	160 ms	107 ms	82.3 ms	116 ms	89.3 ms
4000	5.16 s	5.06 s	2.55 s	1.28 s	860 ms	674 ms	933 ms	731 ms

Table 7: Median time per multiplication, AVX2 build (15 runs).

n	$p=1$	2	4	6	8	12	16
ikj: S(p) and, below, E(p)							
100	0.93	1.53	2.49	2.75	2.84	2.39	2.45
	0.93	0.76	0.62	0.46	0.35	0.20	0.15
250	0.99	1.95	3.66	5.34	6.84	5.65	6.98
	0.99	0.97	0.92	0.89	0.85	0.47	0.44
500	1.00	1.97	3.92	5.88	7.55	5.99	7.81
	1.00	0.98	0.98	0.98	0.94	0.50	0.49
1000	1.00	2.03	4.01	5.98	8.00	8.35	10.22
	1.00	1.02	1.00	1.00	1.00	0.70	0.64
2000	1.00	1.82	3.16	4.11	5.65	7.83	7.39
	1.00	0.91	0.79	0.68	0.71	0.65	0.46
4000	1.00	1.82	3.07	3.70	4.66	6.50	7.56
	1.00	0.91	0.77	0.62	0.58	0.54	0.47
blocked: S(p) and, below, E(p)							
100	0.86	1.46	2.30	2.81	2.91	2.60	2.64
	0.86	0.73	0.58	0.47	0.36	0.22	0.16
250	0.99	1.93	3.71	5.31	6.74	5.52	6.97
	0.99	0.96	0.93	0.88	0.84	0.46	0.44
500	1.01	2.03	3.91	5.81	7.66	5.44	7.07
	1.01	1.02	0.98	0.97	0.96	0.45	0.44
1000	1.03	2.04	4.07	6.03	8.22	5.70	7.57
	1.03	1.02	1.02	1.01	1.03	0.48	0.47
2000	1.02	2.05	4.07	6.11	7.90	5.61	7.28
	1.02	1.02	1.02	1.02	0.99	0.47	0.46
4000	1.02	2.03	4.02	6.00	7.66	5.53	7.06
	1.02	1.01	1.00	1.00	0.96	0.46	0.44

Table 8: Speedup $S(p)$ and, in small type, efficiency $E(p)$, AVX2 build. The scalar values are plotted in Figure 1; all values with bootstrap intervals and the Karp–Flatt metric are in `results/speedup.csv`.

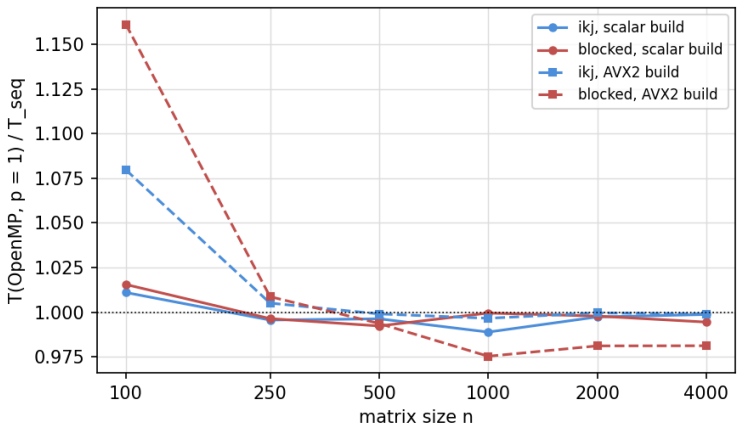


Figure 5: Time of the OpenMP kernel with one thread relative to the sequential kernel.